Health Monitoring System Complete Guide

Last Updated: January 02, 2026

Author: Mehmet Arıkan

Table of Contents

1. System Overview
2. Architecture and Design
3. Prerequisites
4. Installation Guide
5. Admin Panel Features
6. Component Reference
7. Alert Definitions
8. Maintenance and Operations
9. Troubleshooting
10. Network Traffic Analysis

1. System Overview

This is a professional SRE-grade health monitoring system that:

- Monitors 6 critical services (Backend, Frontend, MariaDB, MQTT, Docker, Server Hardware).
- Tracks 71 total alerts across all systems.
- Uses two-tier scraping (always-on and debug) to minimize Prometheus cardinality costs.
- Automatically enables debug metrics during incidents and disables them after resolution.
- Provides fleet-wide health aggregation with status levels (green, amber, red).
- Integrates with Alertmanager for intelligent alert routing and inhibition.

Feature	Description
Dynamic Debug Scraping	Expensive metrics only scraped during incidents (automatic)

Hysteresis Control	10-minute cooldown prevents metric flapping
TTL Protection	Max 30-minute debug window prevents runaway costs
Alert Inhibition	Critical alerts suppress related warnings (reduces noise)
Fleet Health Dashboard	Real-time aggregated status across all services
Blackbox Probing	External TCP/ICMP connectivity monitoring
Professional SLI/SLO	Request latency, error rates, availability metrics

2. Architecture and Design

- **System Components**

The system operates within a Kubernetes cluster (monitoring namespace) and utilizes:

1. Prometheus (KPS Stack): Scrapes metrics every 60s.
2. Alertmanager: Receives firing alerts and routes them via webhooks.
3. Fleet-Health (FastAPI): Aggregates alert data into a unified status.
4. Debug-Scrape-Controller: A Python-based automation that listens for incidents to dynamically inject high-resolution "Debug" ServiceMonitors.

- **Two-Tier Scraping Strategy**
  - Always-On Tier (60s interval): Low-cardinality counters/gauges required for alert evaluation.
  - Debug Tier (15s interval, incident-only): High-cardinality histograms and buckets. Enabled only during incidents.

3. Prerequisites

Infrastructure Requirements:

- Docker: Running with WSL 2 backend.
- Resources: 4+ CPU Cores, 8GB+ RAM (6GB allocated to Minikube).
- Minikube: Version 1.32.0 or higher using the Docker driver.

Software Installation:

1. Install Minikube

winget install Kubernetes.minikube

2. Install kubectl winget

install Kubernetes.kubectl

3. Install Helm

winget install Helm.Helm

Start Minikube with Docker driver and custom resources

minikube start ` --driver=docker ` --cpus=4 ` --memory=6144 ` --disk-size=20g ` --kubernetes-version=v1.28.3 ` --addons=metrics-server

4. Installation Guide

Step 1: Install kube-prometheus-stack

- This installs Prometheus, Alertmanager, and CRDs with Minikube-optimized settings.

Add Prometheus community Helm repository

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

helm repo update

Create monitoring namespace

kubectl create namespace monitoring

Install kube-prometheus-stack

helm install kps prometheus-community/kube-prometheus-stack \

  --namespace monitoring \

  --version 51.0.0 \

  --set prometheus.prometheusSpec.retention=7d \

  --set prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.resources.requests.storage=10Gi \

  --set prometheus.prometheusSpec.resources.requests.memory=1Gi \

```
  --set prometheus.prometheusSpec.resources.limits.memory=2Gi \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false \
  --set alertmanager.enabled=true \
  --set grafana.enabled=false

# Wait for all pods to be ready
kubectl wait --for=condition=Ready pods --all -n monitoring --timeout=600s
```

Step 2: Configure Alertmanager

- Apply custom configuration with webhook routing and inhibition rules.

```
# Apply Alertmanager configuration
helm upgrade kps prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --reuse-values \
  --values deploy/monitoring/am-values.yaml
```

Step 3: Deploy Exporters and Endpoints

- This includes the special network configuration to reach host services from Minikube.

```
# Deploy all exporters (Go to the application directory)
kubectl apply -f deploy/monitoring/2-backend-exporter.yaml
kubectl apply -f deploy/monitoring/2-frontend-exporter.yaml
kubectl apply -f deploy/monitoring/2-mariadb-exporter.yaml
kubectl apply -f deploy/monitoring/2-mqtt-exporter.yaml
kubectl apply -f deploy/monitoring/2-daemon-exporter.yaml
kubectl apply -f deploy/monitoring/2-server-hardware-exporter.yaml
kubectl apply -f deploy/monitoring/2a-docker-events-exporter.yaml

# Deploy local endpoints (connecting to 192.168.65.254)
```

```
kubectl apply -f deploy/monitoring/3-backend-local-endpoints.yaml
kubectl apply -f deploy/monitoring/3-frontend-local-endpoints.yaml
kubectl apply -f deploy/monitoring/3-mariadb-secret.yaml

# Verify all ServiceMonitors
kubectl get servicemonitors -n monitoring
```

Expected Output:

```
NAME                    AGE
backend-app-local       5m
frontend-metrics-local  5m
mariadb-exporter        5m
mosquitto-exporter      5m
docker-daemon           5m
node-exporter           5m
```

Step 4: Deploy Alert Rules

- Deploy PrometheusRule resources for each service.

```
# Deploy all alert rules
kubectl apply -f deploy/monitoring/1-backend-alerts.yaml
kubectl apply -f deploy/monitoring/1-frontend-alerts.yaml
kubectl apply -f deploy/monitoring/1-mariadb-alerts.yaml
kubectl apply -f deploy/monitoring/1-mqtt-alerts.yaml
kubectl apply -f deploy/monitoring/1-docker-alerts.yaml
kubectl apply -f deploy/monitoring/1-server-hardware-alerts.yaml

# Verify PrometheusRules
```

kubectl get prometheusrules -n monitoring

Open http://localhost:9090/rules to check (runs on server.js)

Expected Output:

NAME AGE

backend-app-alerts 10s

frontend-app-alerts 10s

prometheus-mariadb-rules 10s

mqtt-alerts 10s

docker-real-alerts 10s

prometheus-server-hardware-rules 10s

Step 5: Deploy Fleet Health Service

- Deploy the fleet-wide health aggregation service.

Deploy fleet-health service

kubectl apply -f deploy/monitoring/fleet-health.yaml

Verify deployment

kubectl get pods -n monitoring -l app=fleet-health

kubectl get svc -n monitoring fleet-health

Open http://localhost:8080/summary to check (runs on server.js)

Expected Response:

```json

{
```

```
  "status": "green",
  "active": {
    "critical": [],
    "warning": []
  }
}
```


Step 6: Deploy Debug-Scrape Controller

- Deploy the incident-driven debug scraping controller.

```
# Deploy debug-scrape-controller
kubectl apply -k deploy/monitoring/debug-scrape-controller/

# Verify deployment
kubectl get pods -n monitoring -l app=debug-scrape-controller

# Check logs
kubectl logs -n monitoring deploy/debug-scrape-controller -f
```

Expected Log Output:

```
2025-12-31 12:00:00 [INFO] Starting Debug Scrape Controller
2025-12-31 12:00:00 [INFO] Configuration:
  - Alertmanager URL: http://kps-kube-prometheus-stack-alertmanager.monitoring.svc.cluster.local:9093
  - Poll interval: 30s
  - Hysteresis: 10m
  - Max TTL: 30m
```

```
  - Monitored services: ['backend', 'frontend', 'mariadb', 'mqtt', 'docker', 'server-hardware']
2025-12-31 12:00:00 [INFO] ✅ Alertmanager reachable
2025-12-31 12:00:00 [INFO] ✅ All debug ServiceMonitor templates available
```

Step 7: Deploy Blackbox Exporter

- Deploy external connectivity probes.

```
# Install blackbox-exporter
helm install blackbox prometheus-community/prometheus-blackbox-exporter \
  --namespace monitoring \
  --values deploy/monitoring/blackbox-values.yaml

# Deploy probe configurations
kubectl apply -f deploy/monitoring/blackbox-probe-tcp.yaml
kubectl apply -f deploy/monitoring/blackbox-probe-ping.yaml
kubectl apply -f deploy/monitoring/blackbox-internet.yaml
kubectl apply -f deploy/monitoring/blackbox-internet-dns-tcp.yaml
# Verify probes
kubectl get probes -n monitoring
```

5. Admin Panel Health Monitoring Tab

Accessible at http://localhost:3000/admin-panel.

View 1: System Alerts Management

- Real-Time Alert Feed - Live updates from Alertmanager
- Alert History Tracking- Persistent alert timeline with localStorage
- Alert Muting - Mute specific alerts with snooze duration
- Alert Bar Toggle - Floating alert banner for critical alerts

- Export Capabilities - Export alert history as JSON or CSV
- Severity Filtering - Filter by critical/warning/all
- Search - Full-text search across alert names and descriptions
- Alert Statistics- Active, resolved, muted

View 2: System Monitoring Dashboard

- Interactive, dynamic Recharts area charts for all 6 core systems.
- Inline Alert Rule Editor to modify thresholds directly via the UI.

6. Component Reference

- **Backend Service**

Always-On Metrics:

- up: Service availability
- http_requests_total: Request counter by status code, method, and path
- http_request_duration_seconds_bucket: Request latency histogram
- nodejs_process_cpu_seconds_total: CPU usage
- nodejs_process_resident_memory_bytes: Memory usage
- nodejs_nodejs_eventloop_lag_seconds: Event loop lag
- nodejs_nodejs_heap_size_used_bytes: Heap memory used
- nodejs_nodejs_heap_size_total_bytes: Heap memory total

Alert Thresholds (Always-On):

- Backend_Up: Critical if up == 0 for 30s
- Backend_High_Error_Rate: Warning if error rate greater than 5 percent for 5m
- Backend_High_Latency: Warning if p95 latency greater than 500ms for 5m
- Backend_High_CPU: Warning if CPU greater than 80 percent for 5m
- Backend_High_Memory: Warning if memory greater than 85 percent for 5m

- Backend_Heap_Usage_Critical: Critical if heap greater than 95 percent for 2m
- Backend_High_EventLoop_Lag: Warning if lag greater than 100ms for 3m

Debug-Only Metrics (Enabled During Incidents):

- nodejs_nodejs_gc_duration_seconds: Garbage collection timing
- nodejs_nodejs_active_handles_total: Active libuv handles
- nodejs_nodejs_active_requests_total: Active requests
- http_response_size_bytes: Response size histogram

Debug Trigger Alerts:

- Backend_High_Error_Rate (warning severity)
- Backend_High_Latency (warning severity)
- Backend_High_CPU (warning severity)
- Backend_High_Memory (warning severity)

Frontend Service

Always-On Metrics:

- up: Metrics endpoint availability
- prom_react_app_loaded_total: App load success/failure
- prom_react_app_load_time_seconds_bucket: Load time histogram
- prom_react_performance_ttfb_seconds_bucket: Time to first byte
- prom_react_performance_ttfcp_seconds_bucket: Time to first contentful paint
- prom_react_performance_dcl_seconds_bucket: DOM content loaded
- prom_react_ttc_seconds_bucket: Time to content
- prom_react_ttu_seconds_bucket: Time to usable
- prom_react_app_unloaded_total: App unload counter

Alert Thresholds (Always-On):

- Frontend_Up: Critical if up == 0 for 30s

- Frontend_High_Load_Time: Warning if p95 load time greater than 5s for 5m

Debug-Only Metrics:

- prom_react_navigation_duration_seconds_bucket: Navigation timing
- prom_ui_request_seconds_bucket: UI interaction timing

Debug Trigger Alerts:

- Frontend_High_Load_Time (warning severity)

MariaDB Service

Always-On Metrics:

- mysql_up: MySQL exporter connection status
- mysql_global_status_uptime: MySQL uptime
- mysql_global_status_threads_connected: Current connections
- mysql_global_status_threads_running: Active queries
- mysql_global_variables_max_connections: Connection limit
- mysql_global_status_queries: Total queries
- mysql_global_status_open_files: Open file handles
- mysql_global_variables_open_files_limit: File limit
- mysql_global_variables_innodb_force_recovery: InnoDB recovery mode
- probe_success: TCP port reachability (blackbox)

Alert Thresholds (Always-On):

- MariaDB_Down: Critical if mysql_up == 0 for 30s
- MariaDB_Port_Down: Critical if probe_success == 0 for 30s
- MariaDB_High_Connections: Warning if connections greater than 80 percent for 2m
- MariaDB_High_Threads_Running: Warning if running threads greater than 50 for 3m
- MariaDB_High_Open_Files: Warning if open files greater than 80 percent for 2m
- MariaDB_InnoDB_Recovery_Mode: Critical if recovery mode greater than 0

Debug-Only Metrics:

- mysql_global_status_slow_queries: Slow query count
- mysql_global_status_prepared_stmt_count: Prepared statements
- mysql_global_status_innodb_log_waits: InnoDB log waits
- mysql_global_status_innodb_history_list_length: Transaction history

Debug Trigger Alerts:

- MariaDB_High_Threads_Running (warning severity)

MQTT Service

Always-On Metrics:

- up: MQTT exporter availability
- broker_clients_connected: Current connections
- broker_clients_maximum: Max connection limit
- broker_publish_messages_dropped: Dropped messages
- broker_clients_expired: Expired sessions
- broker_retained_messages_count: Retained messages
- probe_success: TCP port reachability (blackbox)

Alert Thresholds (Always-On):

- MQTT_Exporter_Down: Critical if up == 0 for 30s
- MQTT_Port_Down: Critical if probe_success == 0 for 30s
- MQTT_High_Connections: Warning if connections greater than 80 percent for 2m
- MQTT_High_Dropped_Messages: Warning if dropped greater than 10/s for 3m
- MQTT_High_Expired_Clients: Warning if expired rate greater than 5/s for 3m

Debug-Only Metrics:

- broker_bytes_received: Bytes received rate
- broker_bytes_sent: Bytes sent rate

- broker_messages_received: Messages received rate
- broker_messages_sent: Messages sent rate
- broker_subscriptions_count: Active subscriptions

Debug Trigger Alerts:

- MQTT_High_Dropped_Messages (warning severity)

Docker Service

Always-On Metrics:

- up: Docker daemon exporter availability
- engine_daemon_events_total: Docker events counter
- engine_daemon_container_states_containers: Container count by state
- engine_daemon_container_actions_seconds_count: Container action counter
- docker_events_container_total: Container lifecycle events

Alert Thresholds (Always-On):

- Docker_Daemon_Down: Critical if up == 0 for 30s
- Docker_High_Container_Restarts: Warning if restart rate greater than 5 per 5m
- Docker_OOM_Kill: Critical if OOM event detected
- Docker_Data_Root_Full: Critical if disk usage greater than 90 percent

Debug-Only Metrics:

- None: Docker debug tier is disabled (all metrics are inexpensive counters/gauges)

Server Hardware Service

Always-On Metrics:

- up: Node exporter availability
- node_cpu_seconds_total: CPU time by mode
- node_load5: 5-minute load average

- node_memory_MemTotal_bytes: Total memory
- node_memory_MemAvailable_bytes: Available memory
- node_vmstat_oom_kill: OOM killer invocations
- node_filesystem_size_bytes: Filesystem size
- node_filesystem_free_bytes: Filesystem free space
- node_disk_io_time_seconds_total: Disk I/O time
- node_network_up: Network interface status
- node_network_errs_total: Network errors

Alert Thresholds (Always-On):

- Server_Hardware_Down: Critical if up == 0 for 30s
- Server_High_CPU: Warning if CPU greater than 80 percent for 5m
- Server_High_Memory: Warning if memory greater than 85 percent for 5m
- Server_High_Load: Warning if load5 greater than 2 times CPU cores for 5m
- Server_Disk_Full: Critical if disk greater than 90 percent for 2m
- Server_Disk_High: Warning if disk greater than 80 percent for 5m
- Server_High_Disk_IO: Warning if I/O utilization greater than 80 percent for 5m
- Server_Network_Interface_Down: Warning if interface down for 1m
- Server_High_Network_Errors: Warning if error rate greater than 1 percent for 5m
- Server_OOM_Kill: Critical if OOM kill detected

Debug-Only Metrics:

- node_disk_read_bytes_total: Disk read throughput
- node_disk_written_bytes_total: Disk write throughput
- node_network_receive_bytes_total: Network RX throughput
- node_network_transmit_bytes_total: Network TX throughput
- node_pressure_cpu_waiting_seconds_total: CPU pressure stall
- node_pressure_memory_waiting_seconds_total: Memory pressure stall

- node_schedstat_waiting_seconds_total: Scheduler wait time

Debug Trigger Alerts:

- Server_High_CPU (warning severity)
- Server_High_Memory (warning severity)
- Server_High_Load (warning severity)
- Server_High_Disk_IO (warning severity)

Fleet Health Service

Purpose: Aggregates alerts from all services into a single health status.

API Endpoints:

- POST /alert: Receives webhooks from Alertmanager
- GET /summary: Returns fleet health status
- POST /reset: Clears alert history

Status Logic:

- green: No active alerts
- amber: Warning alerts only
- red: At least one critical alert

Debug-Scrape Controller

Purpose: Automatically enables/disables expensive debug metrics during incidents.

Configuration:

- ALERTMANAGER_URL: http://kps-kube-prometheus-stack-alertmanager.monitoring.svc.cluster.local:9093
- POLL_INTERVAL_SECONDS: 30
- HYSTERESIS_MINUTES: 10

- MAX_DEBUG_TTL_MINUTES: 30

7. Alert Definitions

Backend Alerts (17 total)

Alert Name	Severity	Trigger	Duration	Description
Backend_Up	critical	up{job=~".backend."} == 0	1m	Backend is unreachable
Backend_HTTP_Request_Duration_High	warning	P95 latency greater than 500ms	5m	High request latency
Backend_HTTP_Request_Rate_High	warning	greater than 50 req/s	5m	Unexpected traffic spike
Backend_HTTP_Request_Rate_Low	warning	less than 0.01 req/s	30m	Traffic drop (upstream issue)
Backend_HTTP_Error_Rate_High	critical	5xx rate greater than 5 percent	5m	High error rate
Backend_HTTP_5xx_Errors_Count	critical	greater than 1 error/s	5m	Absolute 5xx error count
Backend_HTTP_4xx_Errors_Count	warning	greater than 5 errors/s	5m	Client errors (bad

				requests)
Backend_Requests_Per_Second_Low	warning	less than 0.5 req/s	15m	Sustained low traffic
Backend_Response_Size_Large	warning	P95 size greater than 1MB	5m	Large response payloads
Backend_CPU_Usage_High	warning	CPU greater than 80 percent	5m	High CPU usage
Backend_Memory_Usage_High	warning	Memory greater than 80 percent	5m	High memory usage
Backend_Heap_Usage_High	warning	Heap greater than 80 percent	5m	Heap approaching limit
Backend_Heap_Usage_Critical	critical	Heap greater than 95 percent	2m	Heap critically full
Backend_GC_Duration_High	warning	GC greater than 100ms	5m	Slow garbage collection
Backend_GC_Frequency_High	warning	GC greater than 10/min	5m	Frequent GC cycles
Backend_Event_Loop_Lag_High	critical	Lag greater than 200ms	5m	Event loop blocked
Backend_Active_Handles_High	warning	Handles greater than 1000	5m	Resource leak

Frontend Alerts (11 total)

Alert Name	Severity	Trigger	Duration	Description
Frontend_Metrics_Endpoint_Down	warning	up == 0	5m	Metrics endpoint unreachable
Frontend_App_Load_Failure_Rate_High	critical	Failure rate greater than 5 percent	5m	High app load failures
Frontend_App_Load_Time_High	warning	P95 load time greater than 3s	5m	Slow page loads
Frontend_TTFB_High	warning	P95 TTFB greater than	5m	High time to first byte

		500ms		
Frontend_TTFCP_High	warning	P95 TTFCP greater than 2s	5m	Slow first contentful paint
Frontend_DOMContentLoaded_Slow	warning	P95 DCL greater than 3s	5m	Slow DOM load
Frontend_Navigation_Duration_High	warning	P95 nav greater than 5s	5m	Slow navigation
Frontend_Time_To_Content_High	critical	P95 TTC greater than 10s	5m	Extremely slow content
Frontend_Time_To_Usable_High	warning	P95 TTU greater than 15s	5m	Slow interactivity
Frontend_Unload_Rate_High	warning	Unload rate greater than 10 percent	5m	Users leaving quickly
Frontend_UI_Request_Rate_High	warning	UI requests greater than 50/s	5m	High UI interaction rate

MariaDB Alerts (15 total)

Alert Name	Severity	Trigger	Duration	Description
MariaDB_Node_Down	critical	probe_success == 0	1m	TCP port unreachable
MariaDB_Node_Slow	warning	Query rate less than 0.12/s	1m	Very low query rate
MariaDB_Down	critical	mysql_up == 0	30s	MySQL exporter connection failed
MariaDB_Restarted	info	Uptime less than 60s	0m	Recent restart detected
MariaDB_High_Connections	warning	Connections greater than 80 percent	1m	High connection usage
MariaDB_High_Threads_Running	warning	Running greater than 60 percent	1m	Many active queries

MariaDB_High_QPS	warning	QPS greater than 1000	1m	High query rate
MariaDB_Slow_Queries	warning	Slow queries greater than 0.05/s	1m	Queries exceeding threshold
MariaDB_High_Prepared_Statements	warning	Prepared stmts greater than 1000	5m	Many prepared statements
MariaDB_InnoDB_Log_Waits	warning	Log waits greater than 0	5m	InnoDB log buffer waits
MariaDB_Too_Many_Open_Files	warning	Open files greater than 80 percent	5m	File handle exhaustion
MariaDB_InnoDB_History_Length	warning	History greater than 10000	5m	Long transaction history
MariaDB_InnoDB_Force_Recovery	critical	Force recovery greater than 0	1m	InnoDB in recovery mode
MariaDB_Replication_Lag	warning	Lag greater than 30s	5m	Replication delay
MariaDB_Read_Only_Mode	warning	read_only == 1	5m	Database in read-only mode

MQTT Alerts (6 total)

Alert Name	Severity	Trigger	Duration	Description
MQTT_Exporter_Down	critical	up == 0	2m	MQTT exporter unreachable
MQTT_Broker_TCP_Unreachable	critical	probe_success == 0	2m	MQTT broker port unreachable
MQTT_Connections_Maximum_Reached	critical	Connections greater than 95 percent	5m	Near connection limit

MQTT_Messages_Dropped	critical	Dropped greater than 0	2m	Messages dropped by broker
MQTT_Clients_Expired	warning	Expired greater than 0	5m	Client session expiry
MQTT_Retained_Messages_Growing	warning	Growth greater than 100/10m	10m	Retained message explosion
Alert Name	Severity	Trigger	Duration	Description
MQTT_Exporter_Down	critical	up == 1	2m	MQTT exporter unreachable
MQTT_Broker_TCP_Unreachable	critical	probe_success == 1	2m	MQTT broker port unreachable
MQTT_Connections_Maximum_Reached	critical	Connections greater than 95 percent	5m	Near connection limit

Docker Alerts (9 total)

Alert Name	Severity	Trigger	Duration	Description
Docker_Daemon_Down	critical	up == 0	30s	Docker daemon unreachable
Docker_Daemon_High_Event_Rate	warning	Events greater than 10/s	2m	High Docker event rate
Docker_No_Running_Containers	warning	Running == 0	5m	No containers running
Docker_High_Container_Start_Rate	warning	Starts greater than 2/s	2m	Frequent container restarts
Docker_Container_Bad_ExitCode	critical	Exit code != 0	1m	Container crashed
Docker_Container_OOM_Event	critical	OOM event detected	1m	Container killed by OOM
Docker_Many_Container_Deaths	warning	Deaths greater than	5m	Multiple container

		5/5m		failures
Docker_DataRoot_Disk_Almost_Full	critical	Disk greater than 90 percent	5m	Docker storage full
Docker_Events_Exporter_Down	warning	up == 0	2m	Events exporter unreachable

Server Hardware Alerts (12 total)

Alert Name	Severity	Trigger	Duration	Description
Server_Node_Down	critical	up == 0	2m	Node exporter unreachable
Server_CPU_Usage_High	warning	CPU greater than 80 percent	5m	High CPU usage
Server_CPU_Usage_Critical	critical	CPU greater than 95 percent	2m	Critical CPU usage
Server_Load_Average_High	warning	Load/core greater than 1.5	5m	High load average
Server_Memory_Usage_High	warning	Memory greater than 85 percent	5m	High memory usage
Server_Memory_Usage_Critical	critical	Memory greater than 95 percent	2m	Critical memory usage
Server_Out_Of_Memory	critical	OOM kill greater than 0	1m	OOM killer invoked
Server_Disk_Usage_High	warning	Disk greater than 80 percent	5m	High disk usage
Server_Disk_Usage_Critical	critical	Disk greater than 90 percent	2m	Critical disk usage
Server_Disk_IO_High	warning	I/O time greater than 80 percent	5m	Disk I/O saturation
Server_Network_Interface_Down	critical	node_network_up == 0	2m	Network interface down
Server_Network_High_Error_Rate	warning	Errors greater than 1 percent	5m	High network error rate

8. Maintenance and Operations

Check System Health:

```
# Quick status check
kubectl get pods -n monitoring
kubectl get servicemonitors -n monitoring
kubectl get prometheusrules -n monitoring

# Check fleet health summary
curl http://localhost:8080/summary
```

Check Active Alerts:

1. Port-forward to Alertmanager: kubectl port-forward -n monitoring svc/kps-kube-prometheus-stack-alertmanager 9093:9093
2. Open a web browser and navigate to: http://localhost:9093/#/alerts

Check Prometheus Targets:

1. Port-forward to Prometheus: kubectl port-forward -n monitoring svc/kps-kube-prometheus-stack-prometheus 9090:9090
2. Open a web browser and navigate to: http://localhost:9090/targets

Update Alert Rules

1. Edit the specific alert configuration file: notepad deploy/monitoring/1-backend-alerts.yaml
2. Apply the changes to the cluster: kubectl apply -f deploy/monitoring/1-backend-alerts.yaml
3. Verify that Prometheus has successfully reloaded the rules: kubectl logs -n monitoring prometheus-kps-kube-prometheus-stack-prometheus-0 | grep "Completed loading"

Update Debug Controller

1. Edit the controller Python logic: notepad deploy/monitoring/debug-scrape-controller/debug-scrape-controller.py
2. Redeploy the controller (Kustomize will regenerate the ConfigMap): kubectl apply -k deploy/monitoring/debug-scrape-controller/

3. Restart the deployment to pick up changes: kubectl rollout restart deployment debug-scrape-controller -n monitoring
4. Monitor the logs for initialization: kubectl logs -n monitoring deploy/debug-scrape-controller -f

Adjust Hysteresis and TTL

Option 1: Environment Variables (Recommended) Edit deploy/monitoring/debug-scrape-controller/deployment.yaml:

YAML

```
env:
  - name: HYSTERESIS_MINUTES
    value: "5"
  - name: MAX_DEBUG_TTL_MINUTES
    value: "20"
```

Redeploy changes: kubectl apply -k deploy/monitoring/debug-scrape-controller/

Option 2: Direct Python File Modification Edit deploy/monitoring/debug-scrape-controller/debug-scrape-controller.py:

Python

```
HYSTERESIS_MINUTES = int(os.getenv("HYSTERESIS_MINUTES", "5"))
MAX_DEBUG_TTL_MINUTES = int(os.getenv("MAX_DEBUG_TTL_MINUTES", "20"))
```

Adjust Fleet Health Expiration

Edit line 27 of deploy/monitoring/fleet-health.yaml: EXPIRE_SECONDS = 120 (Changed from 4200)

Redeploy the service: kubectl apply -f deploy/monitoring/fleet-health.yaml kubectl rollout restart deployment fleet-health -n monitoring

Backup Prometheus Data

1. Identify the Persistent Volume Claim (PVC) name: kubectl get pvc -n monitoring
2. Execute a local archive backup: kubectl exec -n monitoring prometheus-kps-kube-prometheus-stack-prometheus-0 -- tar czf /tmp/prometheus-backup.tar.gz /prometheus

3. Copy the backup to the local machine: kubectl cp monitoring/prometheus-kps-kube-prometheus-stack-prometheus-0:/tmp/prometheus-backup.tar.gz ./prometheus-backup.tar.gz

9. Troubleshooting

Problem: Targets Not Showing in Prometheus

Symptom: ServiceMonitor created but the target is missing from the Prometheus UI. Solution:

1. Verify ServiceMonitor existence: kubectl get servicemonitor -n monitoring backend-app-local -o yaml
2. Ensure the Service has matching labels: kubectl get svc -n monitoring -o yaml | Select-String "app: backend-app"
3. Check the Prometheus operator selector: kubectl get prometheus -n monitoring kps-kube-prometheus-stack-prometheus -o yaml | Select-String "serviceMonitorSelector"
4. Apply the required release label: kubectl label servicemonitor backend-app-local -n monitoring release=kps
5. Restart Prometheus to force a refresh: kubectl delete pod -n monitoring prometheus-kps-kube-prometheus-stack-prometheus-0

Problem: Debug ServiceMonitor Not Created

Symptom: Logs show an incident but no high-resolution SM is generated. Solution:

1. Check RBAC permissions: kubectl auth can-i create servicemonitors --as=system:serviceaccount:monitoring:debug-scrape-controller -n monitoring
2. Verify the ServiceMonitor template is accessible in the controller logs.
3. Clean up potentially orphaned or stuck monitors: kubectl delete servicemonitor -n monitoring backend-debug

Problem: Alerts Not Triggering

Symptom: Metric conditions are met but no alert fires in Alertmanager. Solution:

1. Check if the rule is loaded: kubectl get prometheusrules -n monitoring
2. Validate the expression logic in the Prometheus Graph UI (http://localhost:9090/graph).
3. Ensure the release label is present on the rule: kubectl label prometheusrule backend-app-alerts -n monitoring release=kps

Problem: High Prometheus Memory Usage

Symptom: Pod exceeds 4GB RAM or enters CrashLoopBackOff. Solution:

1. Analyze metric cardinality: count({__name__=~".+"}) in Prometheus.
2. Reduce the data retention period: helm upgrade kps prometheus-community/kube-prometheus-stack --namespace monitoring --reuse-values --set prometheus.prometheusSpec.retention=7d
3. Increase memory limits: helm upgrade kps prometheus-community/kube-prometheus-stack --namespace monitoring --reuse-values --set prometheus.prometheusSpec.resources.limits.memory=8Gi

10. Network Traffic Analysis

- Images below represent detailed network analysis conducted on the server during a HIL test with 5 customers at 02.01.2026.

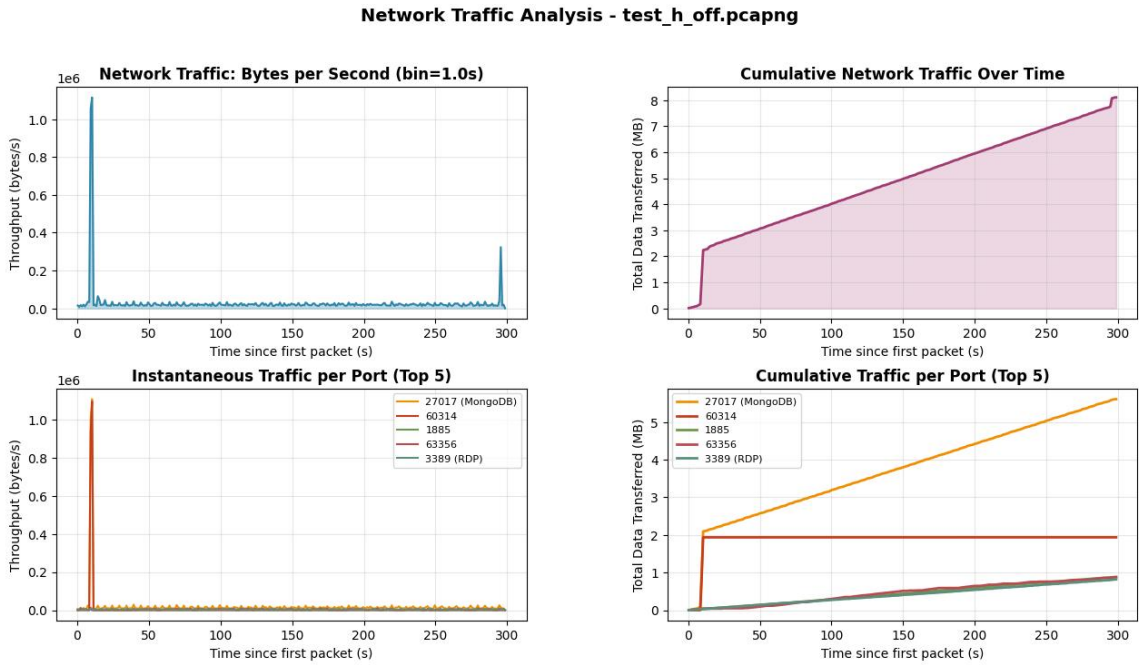

Figure 1. HIL test with health monitoring system is turned off.

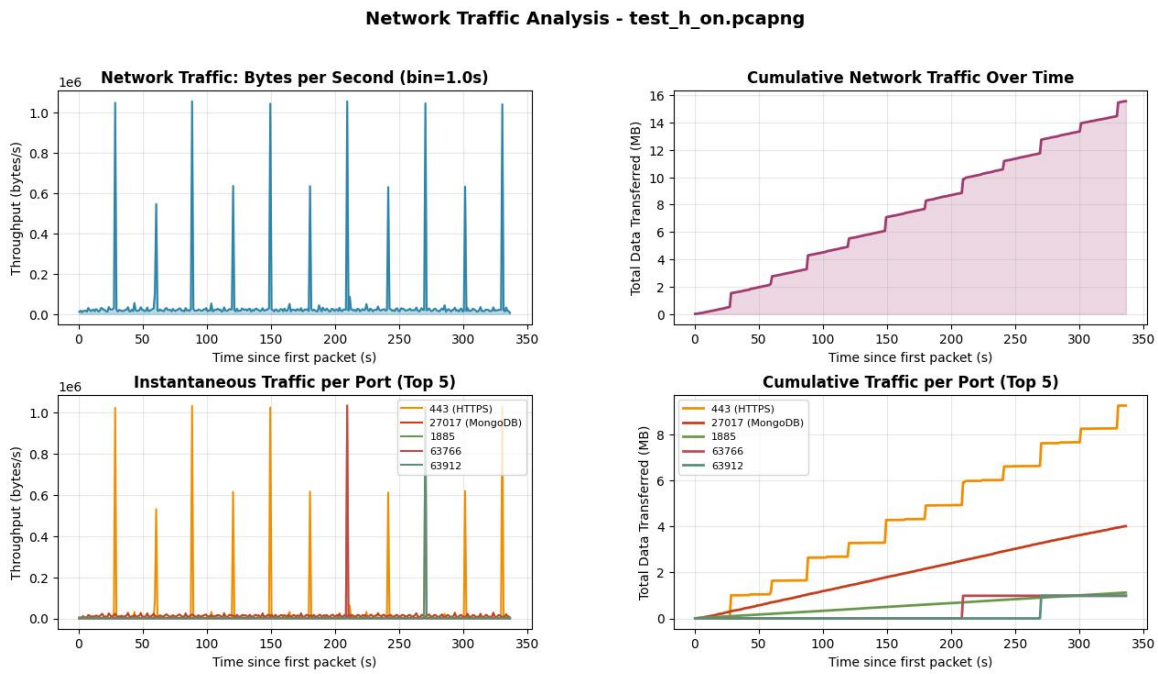

Figure 2. HIL test with health monitoring system is turned on.

It was observed that the health monitoring system only consumed an additional 6 to 8 MB during the same time intervals.